

Створимо бакет

AWS Region

Europe (Stockholm) eu-north-1

Bucket type

Info

General purpose

Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

Directory - New

Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket name

Info

smth147896321

Bucket name must be unique within the global namespace and follow the bucket naming rules. [See rules for bucket naming](#)

Copy settings from existing bucket - optional

Only the bucket settings in the following configuration are copied.

Choose bucket

Format: s3://bucket/prefix

Object Ownership

Info

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

ACLs disabled (recommended)

All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.

ACLs enabled

Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

```
(smth@smth)-[~]
$ aws s3 ls
2024-06-28 13:14:59 smth147896321

(smth@smth)-[~]
$ aws s3 cp /home/smith/Desktop/test.txt s3://smth147896321
upload: Desktop/test.txt to s3://smth147896321/test.txt
```

Бачимо, що файл отримано

Objects (1)

Info

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Find objects by prefix

	Name	Type	Last modified	Size	Storage class
☐	test.txt	txt	June 29, 2024, 04:48:52 (UTC+03:00)	5.0 B	Standard

```
(smth@smth)-[~]  
$ aws s3 cp s3://smth147896321/test.txt /home/smith/Desktop/  
download: s3://smth147896321/test.txt to Desktop/test.txt
```

Скрипт для відправки файлів:

```
GNU nano 8.0                               upl.py  
import logging  
import boto3  
from botocore.exceptions import ClientError  
import os  
  
def upload_file(file_name, bucket, object_name=None):  
    if object_name is None:  
        object_name = os.path.basename(file_name)  
    s3_client = boto3.client('s3')  
    try:  
        response = s3_client.upload_file(file_name, bucket, object_name)  
    except ClientError as e:  
        logging.error(e)  
        return False  
    return True  
  
file_path = '/home/smith/Desktop/test_1.txt'  
bucket_name = 'smth147896321'  
upload_file(file_path, bucket_name)
```

```

(smth@smth)-[~]
$ python3 upl.py
Traceback (most recent call last):
  File "/home/smith/upl.py", line 18, in <module>
    upload_file(file_path, bucket_name)
  File "/home/smith/upl.py", line 8, in upload_file
    object_name = os.path.basename(file_name)
                  ^~~~~~
TypeError: unsupported operand type(s) for -: 'NoneType' and 'str'

(smth@smth)-[~]
$ nano upl.py

(smth@smth)-[~]
$ python3 upl.py
Traceback (most recent call last):
  File "/home/smith/upl.py", line 18, in <module>
    upload_file(file_path, bucket_name)
  File "/home/smith/upl.py", line 9, in upload_file
    s3_client = boto.client('s3')
                ^^^^^
NameError: name 'boto' is not defined. Did you mean: 'boto3'?

(smth@smth)-[~]
$ nano upl.py

(smth@smth)-[~]
$ python3 upl.py

```

Бачимо, що файл було надіслано у бакет

Objects (2) info						
Refresh Copy S3 URI Copy URL Download Open Delete Actions Create folder Upload						
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 Inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more						
Find objects by prefix						
☐	Name	Type	Last modified	Size	Storage class	
☐	 test_1.txt	txt	June 29, 2024, 05:12:18 (UTC+03:00)	5.0 B	Standard	
☐	 test.txt	txt	June 29, 2024, 04:48:52 (UTC+03:00)	5.0 B	Standard	

Додаємо файл через AWS Management Console:

S3 REST API. [Learn more](#)

Drag and drop files and folders you want to upload here, or choose **Add files** or **Add folder**.

Files and folders (1 Total, 0 B)

Remove

Add files

Add folder

All files and folders in this table will be uploaded.

Find by name

< 1 >

☐	Name	Folder	Type
☐	test_2.txt	-	text/plain

Destination [Info](#)

Destination

s3://smth147896321

► Destination details

Bucket settings that impact new objects stored in the specified destination.

Objects (3) [Info](#)



Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 > ⚙

☐	Name	Type	Last modified	Size	Storage class
☐	test_1.txt	txt	June 29, 2024, 05:12:18 (UTC+03:00)	5.0 B	Standard
☐	test_2.txt	txt	June 29, 2024, 05:14:31 (UTC+03:00)	0 B	Standard
☐	test.txt	txt	June 29, 2024, 04:48:52 (UTC+03:00)	5.0 B	Standard

Вмикаємо версіонування через AWS Management Console та виключаємо через AWS CLI

Bucket Versioning

Edit

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#)

Bucket Versioning

Enabled

Multi-factor authentication (MFA) delete

An additional layer of security that requires multi-factor authentication for changing Bucket Versioning settings and permanently deleting object versions. To modify MFA delete settings, use the AWS CLI, AWS SDK, or the Amazon S3 REST API. [Learn more](#)

Disabled

```
(smth@smth)-[~]  
$ aws s3api put-bucket-versioning --bucket smth147896321 --versioning-configuration Status=Suspended
```

Bucket overview

AWS Region Europe (Stockholm) eu-north-1	Amazon Resource Name (ARN)  arn:aws:s3:::smth147896321	Creation date June 28, 2024, 20:14:58 (UTC+03:00)
---	--	--

Bucket Versioning

Edit

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#)

Bucket Versioning
Suspended
Multi-factor authentication (MFA) delete
An additional layer of security that requires multi-factor authentication for changing Bucket Versioning settings and permanently deleting object versions. To modify MFA delete settings, use the AWS CLI, AWS SDK, or the Amazon S3 REST API. [Learn more](#)
Disabled

Скрипт для считування файла та виводу 5 перших рядків:

```
GNU nano 8.0                                downl.py
import boto3
import io

bucket_name = 'smth147896321'
file_name = 'test_3.txt'

s3_client = boto3.client('s3')

try:
    response = s3_client.get_object(Bucket = bucket_name, Key = file_name)
    content = response['Body'].read().decode('utf-8')
    lines = content.split('/n')
    for line in lines[:5]:
        print(line)
except Exception as e:
    print(f"Error: {e}")
```

Результат:

```
(smth@ smth)-[~]
$ aws s3 cp /home/smith/Desktop/test_3.txt s3://smith147896321
upload: Desktop/test_3.txt to s3://smith147896321/test_3.txt

(smth@ smth)-[~]
$ python3 downl.py
File "/home/smith/downl.py", line 5
    file_name 'test_3'
            ^^^^^^^^^
SyntaxError: invalid syntax

(smth@ smth)-[~]
$ nano downl.py

(smth@ smth)-[~]
$ python3 downl.py
File "/home/smith/downl.py", line 5
    file_name 'test_3.txt'
            ^^^^^^^^^^^^^
SyntaxError: invalid syntax

(smth@ smth)-[~]
$ nano downl.py

(smth@ smth)-[~]
$ python3 downl.py
test
idk
smith
i completed my diploma in logistics
yay
```

Скрипт для зміни даних:

```
smth@smth: ~
GNU nano 8.0 data.py
import boto3
import pandas as pd
from io import StringIO

bucket_name = 'smth147896321'
file_name = 'wine.data'
s3_client = boto3.client('s3')

try:
    response = s3_client.get_object(Bucket = bucket_name, Key = file_name)
    content = response['Body'].read().decode('utf-8')
    data = StringIO(content)
    column_names = ['Class', 'Alcohol', 'Malic acid', 'Ash', 'Alcalinity of ash', 'Magnesium', 'Total Phenols',
    df = pd.read_csv(data, names = column_names)

    print("First rows:")
    print(df.head())

    filtered = df[df['Alcohol'] > 13.0]
    print("\nFiltered by Alcohol:")
    print(filtered.head())

    grouped = df.groupby('Class').mean()
    print("\nGroupes by Class:")
    print(grouped.head())

    max = df['Magnesium'].max()
    print("\nMax Magnesium:")
    print(max)

except Exception as e:
    print(f"Error: {e}")
```

```
(smth@ smth)~[~]
$ python3 data.py
First rows:
  Class  Alcohol  Malic acid  Ash  ...  Color intensity  Hue  OD280/OD315 of diluted wine  Proline
0     1    14.23      1.71  2.43  ...           5.64  1.04                3.92      1065
1     1    13.20      1.78  2.14  ...           4.38  1.05                3.40      1050
2     1    13.16      2.36  2.67  ...           5.68  1.03                3.17      1185
3     1    14.37      1.95  2.50  ...           7.80  0.86                3.45      1480
4     1    13.24      2.59  2.87  ...           4.32  1.04                2.93       735

[5 rows x 14 columns]

Filtered by Alcohol:
  Class  Alcohol  Malic acid  Ash  ...  Color intensity  Hue  OD280/OD315 of diluted wine  Proline
0     1    14.23      1.71  2.43  ...           5.64  1.04                3.92      1065
1     1    13.20      1.78  2.14  ...           4.38  1.05                3.40      1050
2     1    13.16      2.36  2.67  ...           5.68  1.03                3.17      1185
3     1    14.37      1.95  2.50  ...           7.80  0.86                3.45      1480
4     1    13.24      2.59  2.87  ...           4.32  1.04                2.93       735

[5 rows x 14 columns]

Groupes by Class:
  Class  Alcohol  Malic acid  Ash  ...  Hue  OD280/OD315 of diluted wine  Proline
1     13.744746   2.010678  2.455593  ...  1.062034                3.157797  1115.711864
2     12.278732   1.932676  2.244789  ...  1.056282                2.785352   519.507042
3     13.153750   3.333750  2.437083  ...  0.682708                1.683542   629.895833

[3 rows x 13 columns]

Max Magnesium:
162
```

```

GNU nano 8.0                                data.py
import boto3
import pandas as pd
from io import StringIO, BytesIO

bucket_name = 'smth147896321'
file_name = 'wine.data'
new_name = 'wine_new.data'
s3_client = boto3.client('s3')

try:
    response = s3_client.get_object(Bucket = bucket_name, Key = file_name)
    content = response['Body'].read().decode('utf-8')
    data = StringIO(content)
    column_names = ['Class', 'Alcohol', 'Malic acid', 'Ash', 'Alcalinity of ash', 'Magnesium', 'Total Phenols', '']
    df = pd.read_csv(data, names = column_names)

    print("First rows:")
    print(df.head())

    filtered = df[df['Alcohol'] > 13.0]
    print("\nFiltered by Alcohol:")
    print(filtered.head())

    grouped = df.groupby('Class').mean()
    print("\nGroupes by Class:")
    print(grouped.head())

    max = df['Magnesium'].max()
    print("\nMax Magnesium:")
    print(max)

    df['Alcohol_Percent'] = df['Alcohol'] / 100
    print(df.head())

    csv_buffer = StringIO()
    df.to_csv(csv_buffer, index = False)
    csv_buffer.seek(0)

    csv_bytes = BytesIO(csv_buffer.getvalue().encode('utf-8'))
    s3_client.put_object(Bucket = bucket_name, Key = new_name, Body = csv_bytes)
    print(f"\nUdated data in {new_name}, {bucket_name}")

except Exception as e:
    print(f"Error: {e}")

```

```

$ python3 data.py
First rows:
  Class Alcohol Malic acid Ash ... Color intensity Hue OD280/OD315 of diluted wine Proline
0      1   14.23      1.71 2.43 ...           5.64 1.04                      3.92   1065
1      1   13.20      1.78 2.14 ...           4.38 1.05                      3.40   1050
2      1   13.16      2.36 2.67 ...           5.68 1.03                      3.17   1185
3      1   14.37      1.95 2.50 ...           7.80 0.86                      3.45   1480
4      1   13.24      2.59 2.87 ...           4.32 1.04                      2.93    735

[5 rows x 14 columns]

Filtered by Alcohol:
  Class Alcohol Malic acid Ash ... Color intensity Hue OD280/OD315 of diluted wine Proline
0      1   14.23      1.71 2.43 ...           5.64 1.04                      3.92   1065
1      1   13.20      1.78 2.14 ...           4.38 1.05                      3.40   1050
2      1   13.16      2.36 2.67 ...           5.68 1.03                      3.17   1185
3      1   14.37      1.95 2.50 ...           7.80 0.86                      3.45   1480
4      1   13.24      2.59 2.87 ...           4.32 1.04                      2.93    735

[5 rows x 14 columns]

Groupes by Class:
  Class Alcohol Malic acid Ash ... Hue OD280/OD315 of diluted wine Proline
1      13.744746  2.010678 2.455593 ... 1.062034 3.157797 1115.711864
2      12.278732  1.932676 2.244789 ... 1.056282 2.785352 519.507042
3      13.153750  3.333750 2.437083 ... 0.682708 1.683542 629.895833

[3 rows x 13 columns]

Max Magnesium:
162
  Class Alcohol Malic acid Ash ... Hue OD280/OD315 of diluted wine Proline Alcohol_Percent
0      1   14.23      1.71 2.43 ... 1.04 3.92 1065 0.1423
1      1   13.20      1.78 2.14 ... 1.05 3.40 1050 0.1320
2      1   13.16      2.36 2.67 ... 1.03 3.17 1185 0.1316
3      1   14.37      1.95 2.50 ... 0.86 3.45 1480 0.1437
4      1   13.24      2.59 2.87 ... 1.04 2.93 735 0.1324

[5 rows x 15 columns]

Udated data in wine_new.data, smth147896321

```

Objects (6) Info						Copy S3 URI	Copy URL	Download	Open	Delete	Actions	Create folder	Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more													
Find objects by prefix													
Show versions													
	Name	Type	Last modified	Size	Storage class								
☐	test_2.txt	txt	05:14:31 (UTC+03:00)										
☐	test_3.txt	txt	June 29, 2024, 05:34:43 (UTC+03:00)	54.0 B	Standard								
☐	test.txt	txt	June 29, 2024, 04:48:52 (UTC+03:00)	5.0 B	Standard								
☐	wine_new.d ata	data	June 29, 2024, 06:32:57 (UTC+03:00)	12.8 KB	Standard								
☐	wine.data	data	June 29, 2024, 05:48:07 (UTC+03:00)	10.5 KB	Standard								

Скрипт для перегляду властивостей об'єктів:

```

import boto3
from botocore.exceptions import NoCredentialsError, PartialCredentialsError

bucket_name = 'smth147896321'
s3_client = boto3.client('s3')

try:
    response = s3_client.list_objects_v2(Bucket = bucket_name)
    if 'Contents' in response:
        objects = response['Contents']
        for obj in objects[:6]:
            print(f"Key: {obj['Key']}")
            print(f"Size: {obj['Size']} bytes")
            print(f>Last Modified: {obj['LastModified']}")
            print("T_T " * 15)

    else:
        print("No objects")

except NoCredentialsError:
    print("Credentials not available")
except PartialCredentialsError:
    print("Incomplete credentials")
except Exception as e:
    print("Error: {e}")

```

```

(smth@smth)-[~]
$ python3 info.py
Key: test.txt
Size: 5 bytes
Last Modified: 2024-06-29 01:48:52+00:00
T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T
Key: test_1.txt
Size: 5 bytes
Last Modified: 2024-06-29 02:12:18+00:00
T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T
Key: test_2.txt
Size: 0 bytes
Last Modified: 2024-06-29 02:14:31+00:00
T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T
Key: test_3.txt
Size: 54 bytes
Last Modified: 2024-06-29 02:34:43+00:00
T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T
Key: wine.data
Size: 10782 bytes
Last Modified: 2024-06-29 02:48:07+00:00
T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T
Key: wine_new.data
Size: 13125 bytes
Last Modified: 2024-06-29 03:32:57+00:00
T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T T_T

```